



CLARUSWAY

WAY TO REINVENT YOURSELF



clarusway



clarusway



clarusway



clarusway



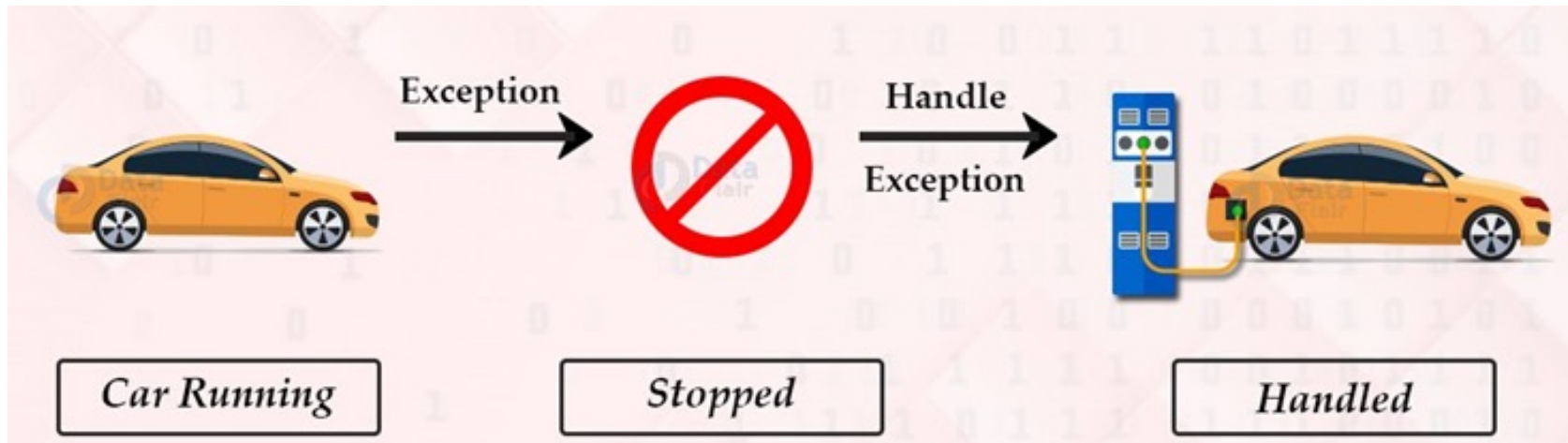
clarusway

Exception Handling

Lesson:
JAVA Chapter 25

Exception

Exception Handling Programs in Java



Exception

Exception programın normal akışında beklenmeyen istisnai durumlara karşılık gelir.

- Dosya okuması yaparken dosyanın harddiskte olmaması hatası,
- Veritabanına bağlanırken bağlantı hatası,
- Web servisi çağırırken bağlantı hatası
- Null bir nesneye erişim hatası gibi

bir çok hatalı durum meydana gelebilir.

Program yazarken belli kurallar yazmış oluruz. Java çalışma ortamı, programımızı bu kurallara göre çalıştırır. Hata ise istisnai (yani kural dışı) bir durumdur. Böyle bir durumda Java çalışma ortamı ne yapacağını bilemez, çareyi programı sonlandırmakta (**stops execution**) bulur.

Bunun önüne geçebilmek için, kodun yazılış aşamasında hata yönetiminin(**throws an exception**) iyi yapılması gerekir.



Exception

Hata yönetimi, hatanın meydana gelmesini engellemek değildir, çalışma sırasında bir hata meydana gelse bile programın çalışmasına devam etmesini sağlamaktır. Java'nın hata yönetim mekanizması sayesinde, program normal akışında çalışır; eğer bir hata olursa, yazdığımız koda uygun olarak bir aksiyon alınır (kullanıcıya hata bilgilendirmesi yapılır, hata kayıt altına alınır vs.) ve sonra program çalışmasına devam eder.

Java'nın **Exception** yönetim mekanizması şu şekilde işler: Programın çalışması sırasında istisnai (**Exception**) bir durum oluşursa bu durumla ilgili bir nesne oluşturulur ve **throws** keyword ile fırlatılır. Böyle bir durumda, programın olağan akışı durdurulur ve bu **Exception**'in yakalanması beklenir.

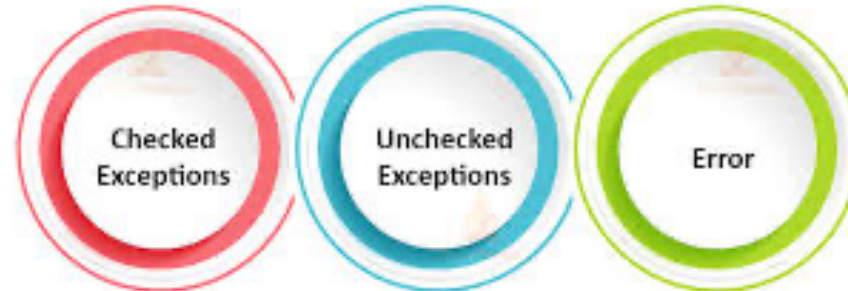
Exception yakalanabilmesi için, **Exception**'a sebep olan kodun **try-catch** bloğu içine yazılması gerekir. Bu durumda Java çalışma ortamı, meydana gelen **Exception** yakalayabilecek bir **catch** bloğu arar, eğer bulursa bu **catch** bloğu çalıştırılır. Son olarak, eğer bir **finally** bloğu yazılmışsa bu blok çalıştırılır ve program normal akışına devam eder.



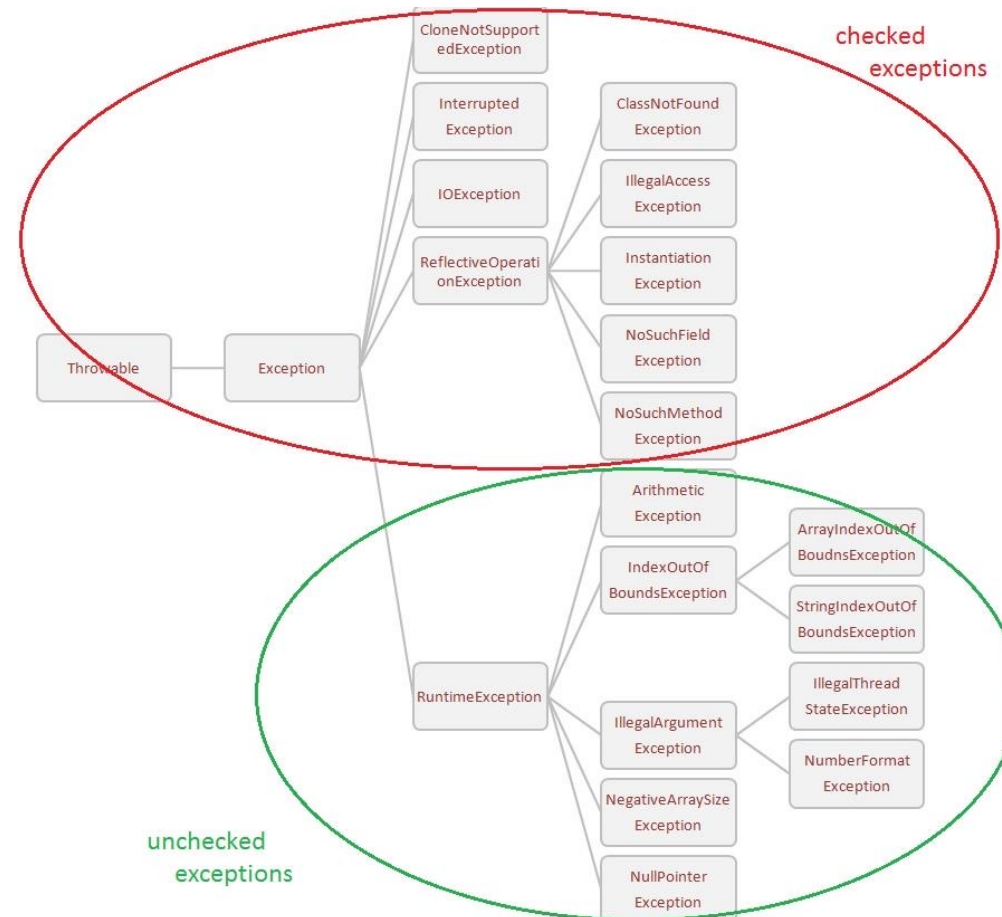
Exception

- **Unchecked (Run Time) Exceptions:** Programın derlenmesi sırasında bilinmeyen ancak program çalıştığı esnada ortaya çıkan exception tipleridir. Çalışma zamanında ortaya çıktıklarından yeniden oluşturulmaları ve tespit edilmeleri daha zordur.
- **Checked (Compile Time) Exceptions:** Derleme aşamasında tespit edilen exception'lardır. Bu hataların alınabileceği önceden bilinir. Örneğin dosyayı açma işlemi Java'da hata oluşturabilecek bir işlemdir. Bu fonksiyonun hata fırlatabileceği önceden belirtildiği için kodu yazarken Java geliştirme platformu ona göre önlem almamızı ister.

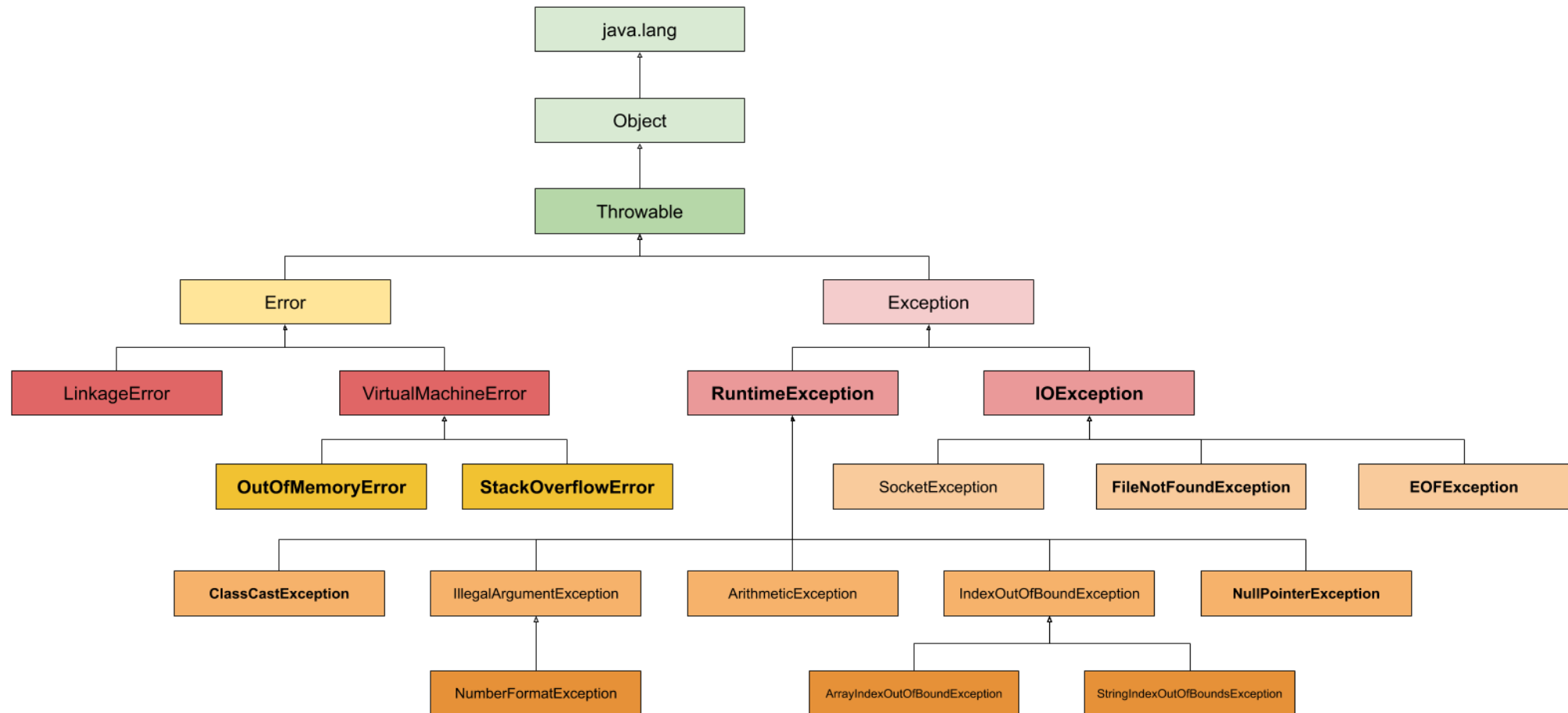
Types of Exceptions in Java



Exception



Exception



Exception

JAVA RunTime'da Exception'ın bilgilerini (type, adres, kaynağını) konsola print eder.

```
class Clarusway {  
    public static void main(String[] args) {  
        int x = 7;  
        int y = 0;  
        System.out.println(x / y);  
    }  
}
```

Exception in thread "main" java.lang.ArithmeticException Create breakpoint : / by zero
at Clarusway.main(Clarusway.java:9)

Exception adres

Exception type

Exception kaynağı



Exception

Exception handle etmek için **try - catch** block create edilir.

*** TRICK :) Try** bloğu tek başına çalışmaz.

Try bloğundan sonra mutlaka **catch block(ları)** veya **finally** block create edilmelidir.

```
class Clarusway {  
  
    public static void main(String[] args) {  
  
        int x = 7;  
        int y = 0;  
        try {  
            System.out.println(x / y);  
        } catch (ArithmeticException e) {  
            System.out.println("Bölme işleminde bölen sıfır değeri alamaz");  
        }  
    }  
}
```

Exception oluşturabilecek code blok

Beklenen Exception type

Gerçekleşen Exception'da çalışacak body



Exception

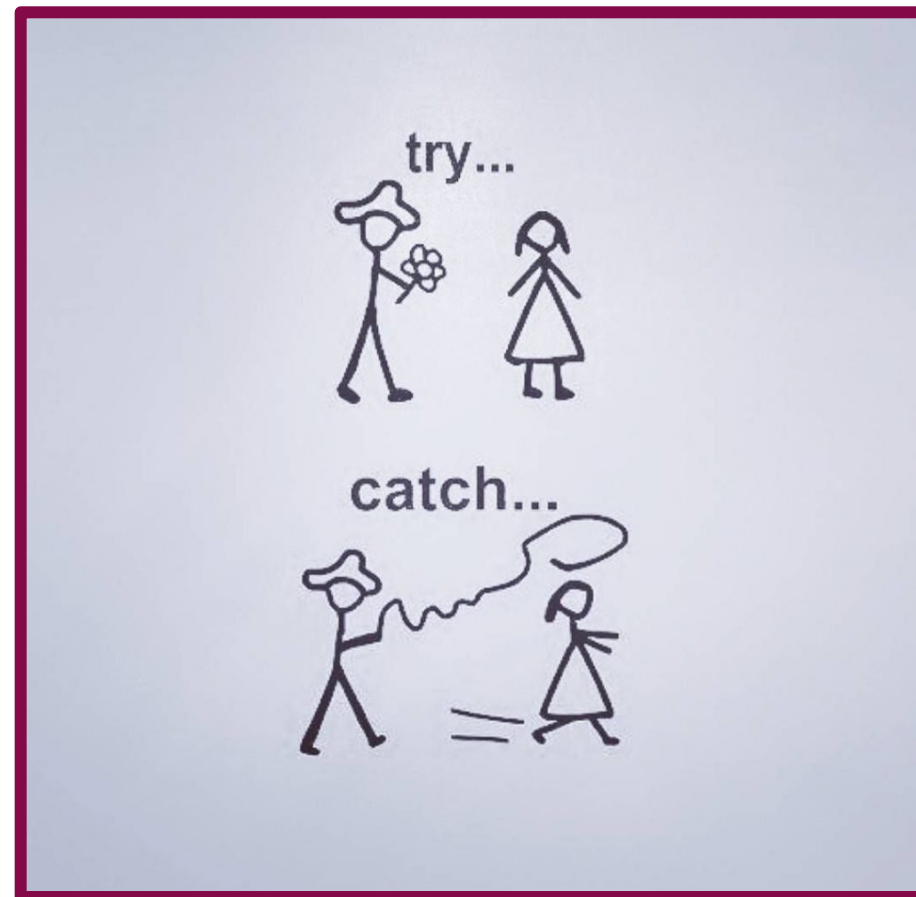
Exception handle etmek için **try - catch** block create edilir.

* **TRICK ;)**

- **try** bloğu tek başına çalışmaz.
try bloğundan sonra mutlaka **catch** block(ları) veya **finally** block create edilmelidir.
- **try** bloğu içerisinde bir **Exception** oluştuğu zaman **Exception'dan** sonraki **try** bloğu içinde kalan kod işletilmez. O noktadan itibaren kodun akışı oluşan **Exception** ile en çok uyumlu olan **catch** bloğuna geçer. O nedenle **catch** bloğunu oluştururken **Exception** tipi seçimi önemlidir. **Catch** bloğuna oluşan **Exception** objesi verilir ve blok içerisinde **Exception** işlenir.
- Eğer hiçbir **Exception** oluşmaz ise kod akışı en son **catch** ifadesinden sonraki kod parçasıyla devam eder. **catch** blokları yalnızca ilgili **Exception** oluştuğu zaman çalıştırılır.
- **try-catch** bloğu kullanımı maliyetlidir. Bu nedenle, "try-catch" bloklarını gerekli yerlerde kullanmak gerekir.



Exception



Exception

```
catch (ArithmeticException e){  
    System.out.printf("Bölme işleminde bölen 0 olamaz"+e.getMessage());  
}
```

Exception Data Type

Variable Name

Bölme işleminde bölen 0 olamaz/ by zero

```
catch (ArithmeticException e){  
    e.printStackTrace();  
    System.out.println("Bölme işleminde bölen 0 olamaz");  
}
```

java.lang.ArithmeticException Create breakpoint : / by zero
at Clarusway.main(Clarusway.java:8)
Bölme işleminde bölen 0 olamaz



Exception

File Input/Output Exceptions

CTE, uyarısı code hatası için değil code run edildiğinde olası okuma hatalarında yapılması gerekenler içindir.

```
class Clarusway {
```

```
    public static void main(String[] args) {
```

```
        FileInputStream file = new FileInputStream( name: "src\\Hardy");
```

```
    }
```

```
}
```

Unhandled exception: java.io.FileNotFoundException

[Add exception to method signature](#) [More actions...](#)

```
public FileInputStream(  
    @NotNull String name  
)  
throws java.io.FileNotFoundException
```

Creates a `FileInputStream` by opening a connection to an actual file, the file named by the path name `name` in the file system. A new `FileDescriptor` object is created to represent this file connection. First, if there is a security manager, its `checkRead` method is called with the name argument as its argument. If the named file does not exist, is a directory rather than a regular file, or for some other reason cannot be opened for reading then a `FileNotFoundException` is thrown.

Params: `name` – the system-dependent file name.

Throws: `FileNotFoundException` – if the file does not exist, is a directory rather than a regular file, or for some other reason cannot be opened for reading.
`SecurityException` – if a security manager exists and its `checkRead` method denies read access to the file.

See Also: `SecurityManager.checkRead(String)`

[java.io.FileInputStream](#)

< 11 >



Exception

Birden çok **Exception** olması durumunda;

1) **Nested** (iç içe)try-catch blokları kullanılabilir.

```
public static void main(String[] args) throws FileNotFoundException {  
    FileInputStream hardyFile = null;  
    try {  
        hardyFile = new FileInputStream( name: "/Users/IdeaProjects/Hardy");  
        int i = 0;  
        try {  
            while ((i = hardyFile.read()) != -1) {  
                System.out.println((char) i);  
            }  
        } catch (IOException ex) {  
            System.out.println("Hardy file okunamıyor");  
            ;  
        }  
    } catch (FileNotFoundException e) {  
        System.out.println("Hardy file'a ulaşılamıyor");  
    }  
}
```



Exception

2) try-multiple catch kullanılabilir.

*** TRICK :) Multiple catch create edilirken parent-child ilişkili olan exception'larda önce child sonra parent Exception create edilmelidir.**

"catch" bloklarının sıralaması önemlidir. Çünkü, sıralanmış haline göre işletilir.

```
public static void main(String[] args) throws FileNotFoundException {  
    FileInputStream hardyFile = null;  
    try {  
        hardyFile = new FileInputStream( name: "/Users/IdeaProjects/Hardy");  
        int i = 0;  
  
        while ((i = hardyFile.read()) != -1) {  
            System.out.println((char) i);  
        }  
    }  
    catch (FileNotFoundException ex) {  
        System.out.println("Hardy file okunamıyor");  
    }  
    catch (IOException e) {  
        System.out.println("Hardy file'a ulaşılamıyor");  
    }  
}
```



Exception

* TRICK :)

Multiple catch yerine
grand-parent Exception
create edilebilir.

```
public static void main(String[] args) throws FileNotFoundException {  
    FileInputStream hardyFile = null;  
    try {  
        hardyFile = new FileInputStream( name: "/Users/IdeaProjects/Hardy");  
        int i = 0;  
  
        while ((i = hardyFile.read()) != -1) {  
            System.out.println((char) i);  
        }  
    }  
    catch (IOException e) {  
        System.out.println("Hardy file okunamıyor veya ulaşılamıyor");  
    }  
}
```



Exception

NullPointerException

Null değeri uygun olmayan variable'de kullanırsa JAVA RunTime'da **NullPointerException** fırlatır.

```
public class Clarusway {  
  
    public static void main(String[] args) {  
        String department = "";  
        System.out.println(department.length()); // 0  
  
        department = null;  
        System.out.println(department.length());  
    }  
}
```

Exception in thread "main" java.lang.[NullPointerException](#) Create breakpoint
at Clarusway.main([Clarusway.java:10](#))



Exception

NumberFormatException

Rakam karakterinden başka karakter bulunduran bir String'i int'e çevirildiğinde JAVA RunTime'da **NumberFormatException** fırlatır.

```
public class Clarusway {  
  
    public static void main(String[] args) {  
  
        String id = "1453571622";  
  
        int invalidID = Integer.parseInt(id);  
        System.out.println(invalidID + 33); // 1453571655  
  
        id = "1453x571622";  
        invalidID = Integer.parseInt(id);  
        System.out.println(invalidID + 33); // RTE  
  
    }  
}
```

```
Exception in thread "main" java.lang.NumberFormatException Create breakpoint : For input string: "1453x571622"  
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)  
    at java.base/java.lang.Integer.parseInt(Integer.java:652)  
    at java.base/java.lang.Integer.parseInt(Integer.java:770)  
    at Clarusway.main(Clarusway.java:13)
```



Exception

ClassCastException

type casting yapılamayacak bir data type işleminde JAVA RunTime'da **ClassCastException** fırlatır.

```
public class Clarusway {  
  
    public static void main(String[] args) {  
  
        Object age = 48;  
  
        String yas = (String) age;  
  
    }  
}
```

```
Exception in thread "main" java.lang.ClassCastException: Create breakpoint : class java.lang.Integer cannot be cast to class java.lang.  
String (java.lang.Integer cannot be cast to java.lang.String)  
at Clarusway.main(Clarusway.java:9)
```



Exception

ArrayIndexOutOfBoundsException

Array veya List'de bulunmayan bir index için işlem yapılırsa JAVA RunTime'da

ArrayIndexOutOfBoundsException fırlatır.

```
public class Clarusway {  
  
    public static void main(String[] args) {  
        String department[] = {"QaTester", "Devops", "Developer"};  
  
        System.out.println(department[2]); // Developer  
  
        System.out.println(department[5]); // RTE  
    }  
}
```

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException Create breakpoint : Index 5 out of bounds for length 3
at Clarusway.main(Clarusway.java:10)



Exception

IllegalArgumentException

Uygulama özelinde bir variable'a istenmeyen değer atandığında handle etmek için RunTime'da **JAVA IllegalArgumentException** fırlatır.

```
public class Clarusway {  
  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Aracınızın km'sini giriniz : ");  
        int km = sc.nextInt();  
        try {  
            if (km < 0) {  
                throw new IllegalArgumentException();  
            }  
        } catch (IllegalArgumentException e) {  
            System.out.println(e);  
            System.out.println("arac km'si negatif olamaz hatalı giriş yaptınız");  
        }  
    }  
}
```

```
Aracınızın km'sini giriniz : -3000  
java.lang.IllegalArgumentException  
arac km'si negatif olamaz hatalı giriş yaptınız
```



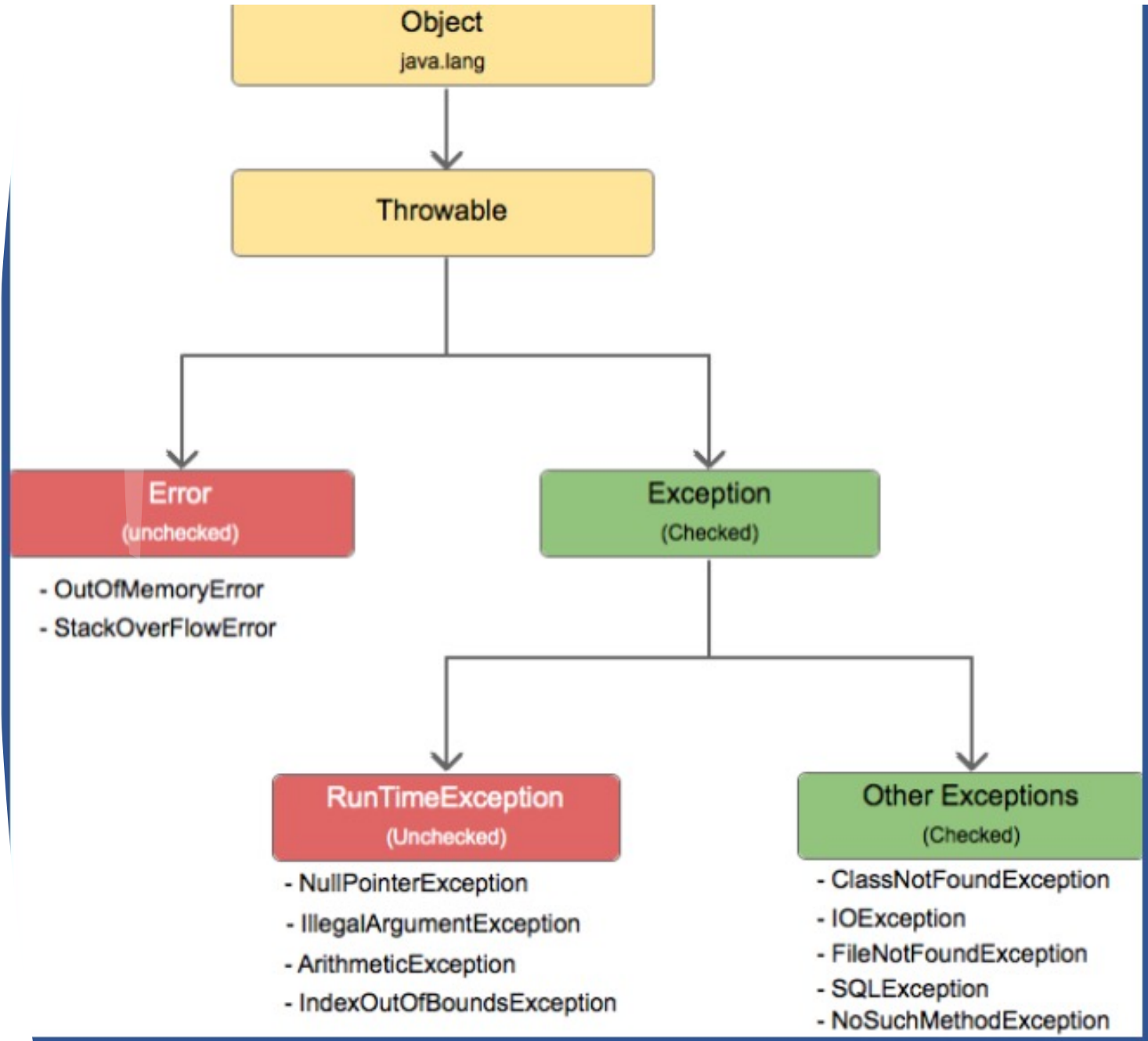
Exception

Throws Keyword

- ❖ throws keyword “checked exceptions” için kullanılır.
- ❖ throws keyword, exception handle edilmeyen durumlarda kullanılır.
- ❖ throws keyword'den sonra, aralarına virgül konularak, birden fazla exception yazılabilir.
- ❖ throws keyword method body'de değil method name satırında yazılmalıdır.
- ❖ throws keyword'den sonra aralarında parent child ilişkisi olan birden fazla exception yazıldığında child exception yazılabilir ama tavsiye edilmez.



Exception



Exception

THROW

- 1- throw keyword kontrollu olarak bir exeption throw etmek için kullanılır
- 2- throw keyword ile sadece bir exeption throw edilebilir
- 3- throw keyword method icinde kullanılır
- 4- throw keyword yazdıktan sonra variable yazılır
(Orn: `new IllegalArgumentException();`)

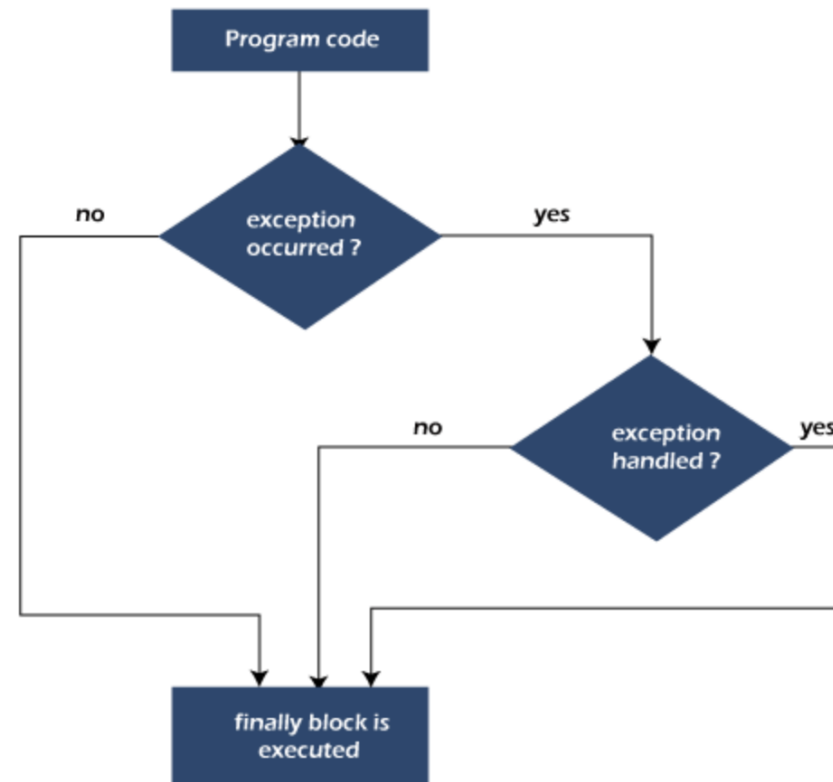
THROWS

- 1- throws keyword bir veya daha fazla exeption'i deklare etmek için kullanılır
- 2- throws keyword bir veya daha fazla exeption throw edilebilir
- 3- throws keyword method signature ile kullanılır
- 4- throws keyword yazdıktan sonra exception class ismi yazılır
(Orn: `FileNotFoundException`)



Exception Finally Block

Flowchart of finally block



Exception Finally Block

- **Finally block** try-catch bloğu ile kullanılır ve try-catch bloğunu takip eder.
- **Finally block**, bir **exception** işlenip işlenmediğine bakılmaksızın her zaman çalışır. Bu nedenle, **exception** olup olmadığına bakılmaksızın yazdırılması gereken tüm gerekli ifadeleri içerir.
- **Finally block** cloud veya database ile connection'ı bitirme veya üzerinde çalışılan dosyayı kapatma gibi işlemler için kullanılır.
- **Finally block** catch bloğu olmadan sadece try ile de kullanılabilir. JAVA throws exception ardından finally ile istediğimiz işlemi yapar.

```
public class Clarusway {  
  
    public static void main(String[] args) {  
  
        int x = 7;  
        int y = 0;  
        try {  
            System.out.println(x / y);  
        } catch (ArithmeticException e) {  
  
            System.out.println("Bölme işleminde bölen 0 olamaz");  
        } finally {  
            System.out.println("cloud ve database connection stop...");  
        }  
    }  
}
```

Bölme işleminde bölen 0 olamaz
cloud ve database connection stop...



Exception Finally Block

Yandaki kod için output nedir?



27

```
String name = "";  
try {  
    name += "Hardy ";  
} catch (Exception e) {  
    name += "Miller ";  
} finally {  
    name += "Java developer ";  
}  
name += "Clarusway ";  
System.out.println(name);
```

Hardy Java developer Clarusway



Exception Finally Block

Yandaki kod için output nedir?



28

```
public static void main(String[] args) {  
    System.out.println(exceptions());  
}  
  
@SuppressWarnings("finally")  
public static String exceptions() {  
    String result="";  
    String v=null;  
  
    try {  
        try {  
            result=result+"a";  
            v.length();  
            result=result+"b";  
        } catch (NullPointerException e) {  
            result=result+"c";  
        } finally {  
            result=result+"d";  
            throw new Exception();  
        }  
    } catch (Exception e) {  
        result=result+"e";  
    }  
    return result;  
}}
```



Exception / Create Custom Checked Exceptions

```
public class ClaruswayEnrolIdCheckedException extends Exception {  
  
    private static final int serialVersionUID = 10001;  
  
    private ClaruswayEnrolIdCheckedException(String message) {  
        super(message);  
    }  
}
```

- Convention Named → Class name sonuna "Exception" kullanılır.
- "checked exception" create etmek için, "Exception" class'ına **extends** edilmelidir.
- "String" parametrelili olan bir constructor'ın ilk satırına **super()**; create edilmelidir.



Exception



OCA Question

Which of the following pairs fill in the blanks to make this code compile? (Choose all that apply)

```
7: public void ohNo() _____ Exception {  
8:     _____ Exception();  
9: }
```

- A. On line 7, fill in throw
- B. On line 7, fill in throws
- C. On line 8, fill in throw
- D. On line 8, fill in throw new
- E. On line 8, fill in throws
- F. On line 8, fill in throws new

Cevap: B, D. In a method declaration, the keyword throws is used. To actually throw an exception, the keyword throw is used and a new exception is created.





Exception

OCA Question

Which exception will the following throw?

```
Object obj = new Integer(3);  
String str = (String) obj;  
System.out.println(str);
```

- A. `ArrayIndexOutOfBoundsException`
- B. `ClassCastException`
- C. `IllegalArgumentException`
- D. `NumberFormatException`
- E. None of the above.

Cevap: B. The second line tries to cast an `Integer` to a `String`. Since `String` does not extend `Integer`, this is not allowed and a `ClassCastException` is thrown.



Exception



OCA Question

What will happen if you add the statement `System.out.println(5 / 0);` to a working `main()` method?

- A. It will not compile.
- B. It will not run.
- C. It will run and throw an `ArithmeticException`.
- D. It will run and throw an `IllegalArgumentException`.
- E. None of the above.

C. The compiler tests the operation for a valid type but not a valid result, so the code will still compile and run. At runtime, evaluation of the parameter takes place before passing it to the `print()` method, so an `ArithmeticException` object is raised.



Errors

Error → Programdaki ciddi hatalı temsil eder.

- JVM tarafından iletilen uygulama dışında oluşan hatalardır.
- Bu tip hatalar da "**Unchecked Exceptions**" tipindedir. Örneğin veritabanı sunucusuna bağlanmaya çalışınca bağlantı hatası verirse bunu ancak RunTime'da anlayabiliriz.

- *** TRICK ;)**

- **Error** daima RunTime'da tespit edilir.
- **Error** ortaya çıktığında uygulama **stops execution** yaparak sonlandırılır.
- **Error** daima **Unchecked'dir**.
- **Error** kesinlikle **handle** edilemez.
- **Error** **throwable class'ın subclass'ıdır**.
- **Error** **Java.lang.Error** paketinde tanımlanırlar.

- Bazı error tanımları:

- ❖ VirtualMachineError
- ❖ Out of Memory Error
- ❖ StackOverflowError
- ❖ System Crash Error



Errors

```
class StackExample {  
    public static void check(int i)  
    {  
        if (i == 0)  
            return;  
        else {  
            check(i++);  
        }  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args)  
    {  
        StackExample.check(5);  
    }  
}
```

Output

```
Exception in thread "main" java.lang.StackOverflowError  
    at StackExample.check(Main.java:7)  
    at StackExample.check(Main.java:7)  
    at StackExample.check(Main.java:7)  
    at StackExample.check(Main.java:7)  
    at StackExample.check(Main.java:7)  
    at StackExample.check(Main.java:7)  
    at StackExample.check(Main.java:7)  
    at StackExample.check(Main.java:7)  
    at StackExample.check(Main.java:7)  
    at StackExample.check(Main.java:7)  
    at StackExample.check(Main.java:7)  
    at StackExample.check(Main.java:7)
```



Karşılaştırma Tablosu

35

Karşılaştırma için temel	Hata	İstisna
Temel	Hata, sistem kaynaklarının eksikliğinden kaynaklanır.	Kod nedeniyle bir istisna ortaya çıkıyor.
Kurtarma	Bir hata düzeltilemez.	Bir istisna kurtarılabilir.
Anahtar kelimeler	Bir hatayı program koduyla çözmenin bir yolu yoktur.	İstisnalar, "try", "catch" ve "throw" üç anahtar sözcüğü kullanılarak ele alınır.
sonuçlar	Hata tespit edildiğinde program anormal şekilde sonlandırılır.	Bir istisna tespit edildiğinde, karşılık gelen "atma" ve "yakalama" anahtar sözcükleri tarafından atılır ve yakalanır.
Türleri	Hatalar denetlenmeyen tür olarak sınıflandırıldı.	İstisnalar kontrol edilmiş veya kontrol edilmemiş tip olarak sınıflandırılır.
paket	Java'da hatalar "java.lang.Error" paketi olarak tanımlanmıştır.	Java'da, bir istisna "java.lang.Exception" içinde tanımlanmıştır.
Örnek	OutOfMemory, StackOverflow.	İşaretili İstisnalar: NoSuchMethod, ClassNotFoundException. İşaretilenmemiş İstisnalar: NullPointerException, IndexOutOfBoundsException.

Errors



Garbage Collector



Garbage



Finalized Garbage



Object Destroyed

Garbage Collection (Çöp Toplama) İşlemi

Garbage Collection, otomatik bellek yönetimi mekanizmasıdır. Bu işlem heap belleğe bakıp, kullanılan objelerin tespit edilmesi ve referans edilmeyenlerin silinmesi üzerine kuruludur. Kullanılmayan/referans edilmeyen nesnelerin kapladığı alan bellekte boşa çıkarılır ve bellekte boş yer açılmış olur. Bu işlemi yapan mekanizmaya da **Garbage Collector** denir.

İşaretle: Kullanılan ve kullanılmayan objeler işaretlenir.

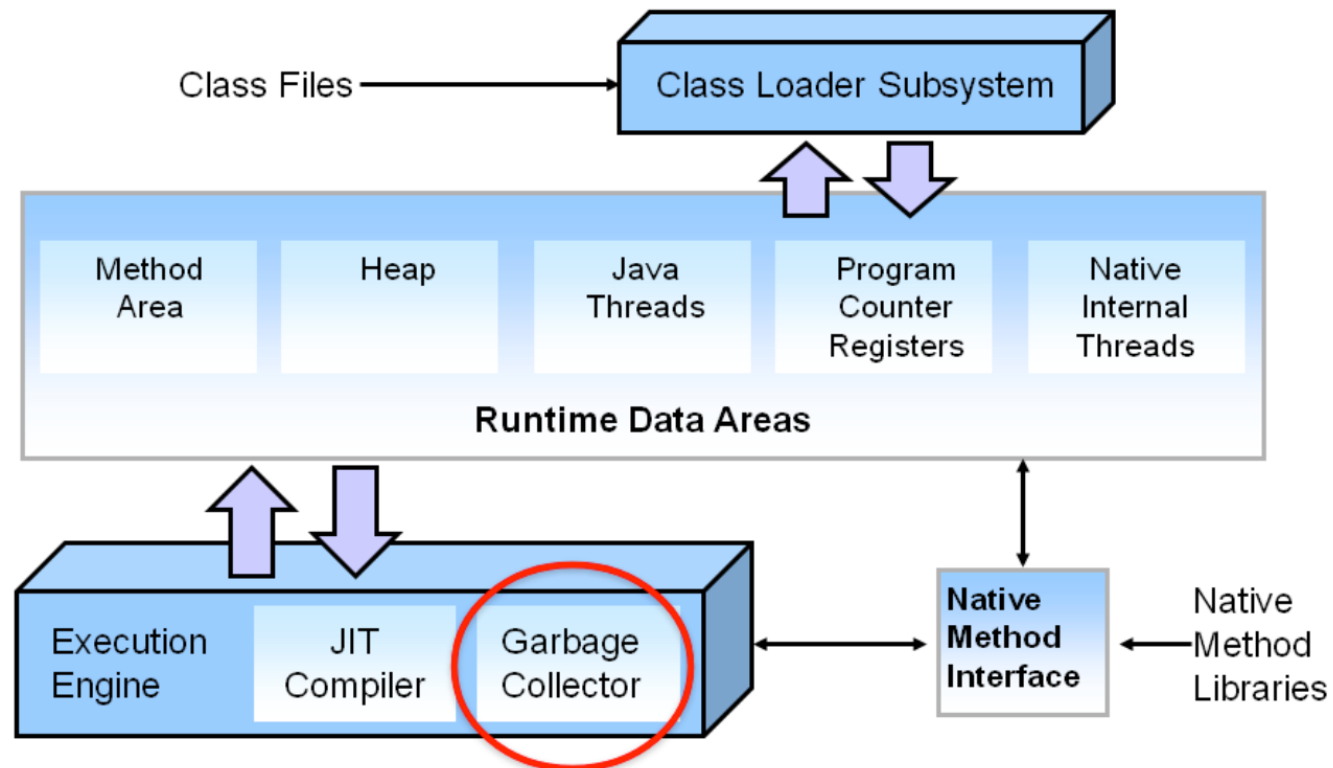
Silme(Destroy): Referansı olmayan objeler heap alanından temizlenir.

Sıkıştırma ve Düzenleme: Silme işleminden sonrae daha büyük objelere boş alan oluşturmak için kalan objeleri bir araya getirilir.



Garbage Collector

HotSpot JVM: Architecture



final – finally – finalize()

Java Final Keyword

- ⇒ Stop Value Change
- ⇒ Stop Method Overriding
- ⇒ Stop Inheritance



final – finally – finalize()

final → keyword
finally → code blok
finalize() → method

- **final keyword**
 - ❖ **Final variable** → Değeri değiştirilmesi istenmeyen (**constant**) **variable'lar** için kullanılır. Final variable'lara ilk değer ataması (**initilaze**) mutlaka yapılmalı yoksa CTE verir. Named convention olarak büyük harf kullanılır.
 - ❖ **Final method** → **Override** edilmesi istenmeyen **method'lar** için kullanılır.
 - ❖ **Final Class** → **Inherit** (kalıtım) edilmesi istenmeyen **class'lar** için kullanılır.
- **finally code blok**

try-catch bloğu ile kullanılır ve try-catch bloğunu takip eder. bir **exception** işlenip işlenmediğine bakılmaksızın her zaman çalışır. Bu nedenle, **exception** olup olmadığına bakılmaksızın yazdırılması gereken ifadeleri içerir.
- **finalize() method**

inalize() method Garbage Collector tarafından silinmesi gereken objeleri imha edilmeden önce çalıştırılır. Garbage Collector sadece finalized yapılmış objeleri toplar ve yok eder.



final – finally – finalize()

```
class Bike9{  
    final int speedlimit=90; //final variable  
    void run(){  
        speedlimit=400;  
    }  
    public static void main(String args[]){  
        Bike9 obj=new Bike9();  
        obj.run();  
    }  
} //end of class
```

Output: Compile Time Error



final – finally – finalize()

```
class Bike{  
    final void run(){System.out.println("running");}  
}  
  
class Honda extends Bike{  
    void run(){System.out.println("running safely with 100kmph");}  
  
    public static void main(String args[]){  
        Honda honda= new Honda();  
        honda.run();  
    }  
}
```

Output:Compile Time Error



final – finally – finalize()

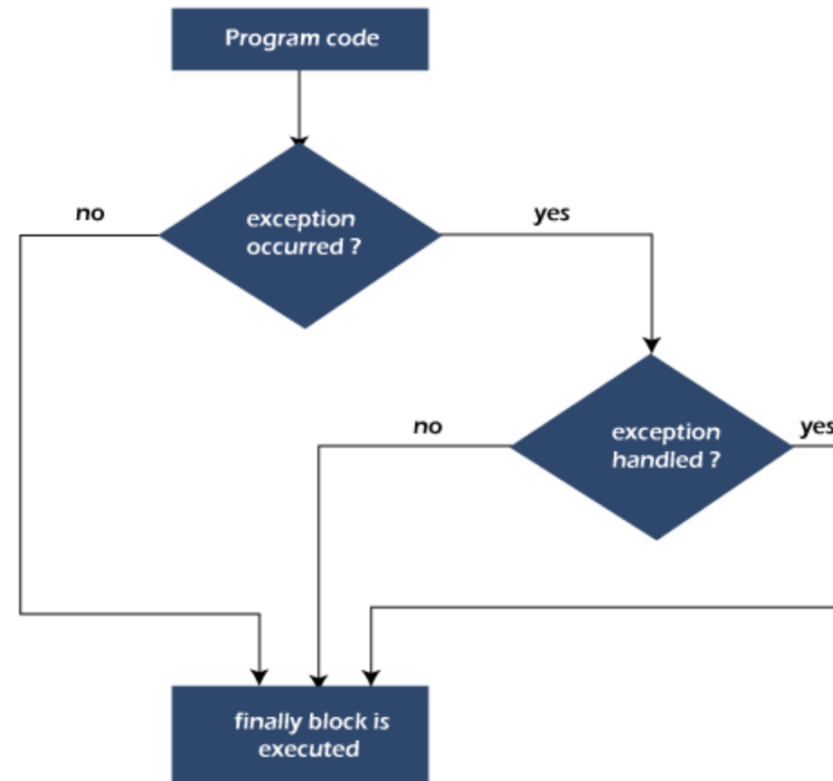
```
final class Bike{  
  
class Honda1 extends Bike{  
    void run(){System.out.println("running safely with 100kmph");}  
  
    public static void main(String args[]){  
        Honda1 honda= new Honda1();  
        honda.run();  
    }  
}
```

Output:Compile Time Error



finally Block

Flowchart of finally block



finally Block

- **Finally block** try-catch bloğu ile kullanılır ve try-catch bloğunu takip eder.
- **Finally block**, bir **exception** işlenip işlenmediğine bakılmaksızın her zaman çalışır. Bu nedenle, **exception** olup olmadığına bakılmaksızın yazdırılması gereken tüm gerekli ifadeleri içerir.
- **Finally block** cloud veya database ile connection'ı bitirme veya üzerinde çalışılan dosyayı kapatma gibi işlemler için kullanılır.
- **Finally block** catch bloğu olmadan sadece try ile de kullanılabilir. JAVA throws exception ardından finally ile istediğimiz işlemi yapar.

```
public class Clarusway {  
  
    public static void main(String[] args) {  
  
        int x = 7;  
        int y = 0;  
        try {  
            System.out.println(x / y);  
        } catch (ArithmeticException e) {  
  
            System.out.println("Bölme işleminde bölen 0 olamaz");  
        } finally {  
            System.out.println("cloud ve database connection stop...");  
        }  
    }  
}
```

Bölme işleminde bölen 0 olamaz
cloud ve database connection stop...



THANKS!

Any questions?

HalUk Bilgin | JAVA Backend Developer

BAŞARI GAYRETE AŞIKTIR 😊

